ABET App: Final Technical Report

Caleb Jones, Emmet Larson, Olivia Hornbeck, Caroline Young, Jonathan Waight

May 6, 2026

Abstract

Engineering and Technology programs in colleges and universities across the world are accredited by an organization called ABET, and maintaining this accreditation is an important process for the institutions and the alumni involved. ABET accreditation has many requirements, but one requirement in particular requires faculty to collect, format, and evaluate data over subsequent six-year stretches of time. The ABET App is a web application which began development during the fall semester of 2025 at York College of Pennsylvania, created with the purpose of simplifying the accreditation process. The ABET App acts as a place for storing relevant data, organizing the collection process, and automatically formatting the data for submission.

Introduction

ABET is a non-profit organization that provides accreditation for academic programs within the fields of science, technology, engineering, and mathematics (STEM). ABET accreditation is important for both institutions and alumni because it shows that the institution meets professional standards and that the alumni are capable of the work ahead of them, while giving institutions a process for facilitating the growth of their program. As of November 2025, ABET reports on the home page of its website that they have accredited over four thousand programs across forty-two different countries[1]. Maintaining accreditation is a core obligation of every accredited institution, and it is a complicated process with many different requirements. One of these requirements is the assessment process, in which every six years an accredited institution must submit a report demonstrating how the program has adapted to better meet the requirements set by ABET. This task requires specific formatting, data management, and careful coordination among the program's faculty.

During the spring semester of 2025 at York College of Pennsylvania (YCP), a team was given the opportunity by Dr. David S. Babcock, a professor and administrator within the YCP Computer Science program, to design and create a project to simplify the assessment process. This project was the first iteration of the ABET App, and although this work was not ultimately used, it served as a foundation for what was to come. During the fall semester of the same year, we were given the opportunity by Dr. Babcock to continue this project, and to expand its scope to better solve the challenge presented to us. This project has continued through the Spring 2026 semester. The ABET App, in its current iteration, is an open source web application designed to simplify the assessment process by storing the relevant data, providing administration with the means to better monitor the data, and using the data to automatically format documentation for the finalized report.

Background

ABET assessments provide accredited institutions with a structured mechanism to measure student learning outcomes, evaluate program effectiveness, and monitor long-term curricular improvement. These assessments also serve as formal evidence to ABET that the institution's engineering and technology programs continue to meet established accreditation criteria and program objectives. ABET's goals in this context are formally referred to as "Student Outcomes"[2]. To properly describe the assessment process used by accredited institutions, it is first necessary to define this term, along with several related terms that are central to ABET's continuous improvement framework. The following terms are relevant to this project:

Student Outcome: "What students are expected to know and be able to do by the time of graduation. These relate to the knowledge, skills, and behaviors that students acquire as they progress through the program."[2]

Assessment: "One or more processes that identify, collect, and prepare data to evaluate the attainment of student outcomes. Effective assessment uses relevant direct, indirect, quantitative and qualitative measures as appropriate to the outcome being measured. Appropriate sampling methods may be used as part of an assessment process."[2]

Evaluation: "One or more processes for interpreting the data and evidence accumulated through assessment processes. Evaluation determines the extent to which student outcomes are being attained. Evaluation results in decisions and actions regarding program improvement."[2]

Performance Indicator: A type of assessment utilized by many accredited institutions, including York College of Pennsylvania. Performance indicators correspond to individual student outcomes. Performance indicators are usually defined by an abstract goal, i.e. "Students can correctly interpret a computational problem and define its parameters." Each Student outcome can contain many corresponding performance indicators.

Measure: A type of assessment used to provide data on performance indicators. Measures are typically defined by a specific task or observation, i.e. a specific question on an exam or a homework assignment. Each performance indicator may be linked to several different measures.

FCAR: An acronym which stands for Faculty Course Assessment Report. In the context of the ABET App, an FCAR is simply a completed measure that has been properly formatted.

Recommended Action: A set of actions that an institution will take going forward to improve the effectiveness of their program by better achieving student outcomes.

Assessment schedule

Outcome	2024-2025	2025–2026	2026–2027	2027–2028	2028–2029	2029–2030
1	(*350*, **400**)	(*101*, *201*, **360**)	(*320*, *350*, **400**)	(*201*, **360**, **402**)	(*101*, *320*, *350*, **400**)	(*201*, *320*, **360**, **402**)
2	(*320*, *330*)	(*335*, **360**, **400**)	(201, 330)	(*320*, *335*, **360**, **402**)	(*330*)	(*201*, *330*, **360**, **402**)
3	(**400**)	(*320*)	(*320*, **400**)	(**402**)	(**400**)	(*320*, **402**)
4	(*330*)	(*335*, **456**)	(330)	(*335*, **456**)	(*330*)	(*335*, **456**)
5	(**400**)	(*320*)	(*320*, **400**)	(**402**)	(**400**)	(*320*, **402**)
6	(*201*, **420**)	(*101*, *320*, *335*, **340**, **400**)	(*201*, **420**)	(*101*, *335*, **340**, **402**)	(*101*, *320*, **420**)	(*201*, *335*, **340**, **402**)

Course	1	2	3	4	5	6
CS 101	1, 5					1
CS 201	1, 3, 5	5				1, 2, 3
CS 320	2, 4	1, 2, 3, 4	1, 2, 3, 4		1, 2, 3	2, 4, 6
CS 330		6, 7		1		
CS 335		6, 7		1		5
CS 340						1
CS 350	1, 3, 5					
CS 360	1, 3, 5, 6	5				
CS 400	2, 4	1, 2, 3, 4	1, 2, 3, 4		1, 2, 3	4, 6
CS 402	2, 4	1, 2, 3, 4	1, 2, 3, 4		1, 2, 3	4, 6
CS 420						1, 3, 5
CS 456				1, 2		

Figure 1. Assessment Schedule

At YCP, assessments are performed every semester based on an assessment schedule (shown in Figure 1). Based on a course mapping, each course that has been scheduled for assessments is assigned corresponding performance indicators, and by extension, corresponding student outcomes. These performance indicators each require the instructor or instructors to conduct one or more measures. The data for the measure could be determined by the instructor's observations, performance on assignments, or the performance on specific parts of assignments. One common example would be performance on an individual exam question. These measures each need to be formatted by the instructor into an FCAR, and turned in to an administrator within the program. Administration within the program must then ensure that every FCAR is securely stored for future use. At the end of a year, the faculty within a program meet for an annual assessment retreat, during which the data for the year is organized into tables, referred to as data summaries, arranged first in order of the student outcomes, and then in order of the performance indicators. Figure 2 is an example of one such data summary. The faculty then

performs an evaluation of the data, determining the degree to which each student outcome has been achieved, any recommended actions to be taken, and whether or not the program would benefit from any changes being made to the performance indicators.

Student Outcome (1) Data Summary

Performance Indicator	Course/Semester Assessed	Assessment Tool or Attribute	Meets/Exceeds Expectation	Target Threshold
2018-2019 – Met **Action:** Monitor				
1.1	CS350, Fa 2018	Assignment 3	77.0%	70%
	CS360, Sp 2019	Exam 1-Takehome, Question 3a	83.3%	70%
	CS360, Sp 2019	Exam 2-Inclass, Question 1	58.6%	70%
1.2	CS350, Fa 2018	Final, Prog Prob 2	68.4%	70%
	CS360, Sp 2019	Exam 2-Inclass, Question 2	69.0%	70%
	CS360, Sp 2019	Exam 4-Inclass, Question 1	64.3%	70%
1.3	CS350, Fa 2018	Exam 2-Question 7	68.5%	70%
	CS360, Sp 2019	Exam 1-Takehome, Question 1b	86.7%	70%
	CS360, Sp 2019	Exam 4-Takehome, Question 2	71.4%	70%
1.4	CS320, Sp 2019	Assignment 5, Use Cases	100.0%	60%
	CS481, Fa 2018	Assignment 3, Requirements	84.0%	80%
1.5	CS320, Sp 2019	Assignment 6, Prob Domain/UML	95.7%	60%
	CS481, Fa 2018	Assignment 4, Analysis and Design	84.0%	80%

Figure 2. Student Outcome Data Summary

Every six years marks the ABET Accreditation Cycle, in which the data from the previous six annual retreats must be reorganized and presented to ABET. The data summaries are still organized according to student outcome above all else, meaning that the first data summary will contain the data for the first student outcome over the course of six years, followed by a narrative on each of the recommended actions pertaining to the student outcome taken throughout the accreditation cycle, which will then repeat for every following student outcome.

This is a complicated process that requires careful planning and coordination amongst the faculty, as well as the secure storage of data over a period of many years. The purpose of the ABET App is to lessen the burden of this process on the faculty involved by accomplishing the following tasks:

- Store the data in a secure, centrally accessible location
- Simplify the submission process of measure data for an instructor through a web application
- Simplify the collection process of measure data for an administrator via a dashboard in the application
- Automatically format the data into data summaries

Technical Terms

The following technical terms are relevant to this project:

API: An acronym which stands for “Application Programming Interface”. A software that allows for multiple computer programs to interact with each other.

CRUD: An acronym which stands for “Create Read Update Delete”. This refers to the four basic operations for persistent storage within computer programming.

Class: In computer programming, a class is a structure used to define an object. It is often described as a blueprint; like a blueprint, it contains the steps for how to build an object.

Dependency: A dependency is a package that is required to be installed in order for a different package to function correctly.

Method: In computer programming, a method is an action that can be performed by an object.

Instance: A term that is typically used to refer to a specific continuous occurrence of a class in computer programming. Throughout this report, it will also be used to refer to a specific continuous occurrence of the ABET App running.

Framework: A tool that provides pre-made code for solving specific problems in computer programming, allows for faster development.

IDE: An acronym which stands for “Integrated Development Environment". An application designed to help software developers program with an integrated set of tools.

Open Source: Denotes that a software project is free to use, distribute, and modify.

Package: In terms of a software development environment, a package is an external library that is utilized to perform specific functions.

Frontend: The part of a computer program that users will typically engage with directly. For example, the homepage of a website would be considered part of that website’s frontend.

Backend: The part of a computer program that users do not typically engage with directly. For example, the database that stores user login information on a social media site would be considered part of the backend.

Database Schema: A structural blueprint for a database, the part of a program where data is stored.

Relational Database: A type of database in which the data is organized into different tables, which each have specific relationships with each other in the context of the tables being connected.

Table: Within the context of a relational database, a table is a unit for organizing, and storing data in the database. Each value that is contained within a table is referred to as a field, and each of these fields has different constraints applied to them to only allow specific data.

Entry: A collection of data that has been categorized by a table. For example, if a user created an account on a website, their account information would be stored within an entry of a table typically called something similar to "user" or "customer".

Junction Table: A table that exists primarily for the purpose of establishing a relationship between two or more tables.

Container: Similar to a virtual machine, a container is a computation space in a Docker environment. It allows a program to run and execute in a consistent environment that can be shared and deployed with other developers.

Database Changelog: A series of files containing statements that modify the existing database schema while keeping an organized history of previous database states.

Technology Used

Visual Studio Code: An IDE used by the team to develop the project. Visual Studio Code was chosen because we each already had experience in using it, and because it has a vast library of extensions which can be downloaded directly from the program itself. Furthermore, it is important for any team of developers to have a similar working environment to each other as it allows the team to better communicate the issues they are facing.[3]

Node.js (^20.19.0): "An Asynchronous, event-driven, Java/TypeScript runtime, designed to build scalable applications."[4]

Node Package Manager (11.6.0): An open source software registry designed to handle packages from Node.js. It allows for version management of dependencies, can automatically

resolve issues with vulnerabilities in dependencies, and is also used to run the frontend development server.[5]

Gradle (9.2.0): An automated build tool with JavaScript support. Used for deployment of the entire project, and for running test methods. The reason Gradle was specifically chosen for the project is because Gradle is system agnostic, meaning that even though the project was developed on mainly on Windows machines it will still function on a Mac or Linux machine.[6]

Spring Boot (3.2.0): An open source Java web framework used to develop the backend of the project. Spring Boot includes many features and plugins that have seen extensive use within the backend, ranging from validating incoming data to encrypting user passwords.[7]

MariaDB (12.0.2): An open source, relational database management system used for the project. It was chosen specifically because it's a modified version of a relational database that the team was already familiar with, it's open source, and because it has additional functionality that's useful for the project in question, such as native support for system-versioned tables, and thread pooling.[8]

Vue.js (^3.5.18): An open source, frontend framework used in the project for developing the user interface.[9]

Vite (^7.0.6): An open source, frontend build tool characterized as being fast, highly optimized, and compatible with many different languages including Java/TypeScript. This tool is used within the project to build the ABET App's frontend.[10]

JUnit (5): An open source, testing framework for Java/TypeScript. Used in the project to create backend tests.[11]

Playwright (^1.54.1): An open source framework for end-to-end testing. It can simulate user interactions and looks for certain designated markers in the user interface to make sure it responds in the expected way. Used within the ABET App for the purposes of testing a user experience from start to finish within the frontend design.[12]

H2 (2.4.240): An application which allows for a SQL database to be created in memory at runtime. This application is used as a way of testing the database interactions, debugging, and for demonstration purposes. The temporary nature of the database makes it well suited for these types of tasks.[13]

Lucidchart: A web application used for the creation of technical diagrams. The team used this tool to create the project's UML Diagram, and to design the project's database schema.[14]

Jakarta Validation (3.1.1): A validation model compatible with Spring Boot, used in the project to apply constraints to specific data before it is moved to or from the database.[15]

Tailscale (1.90.9): A VPN service that has been integrated into the CI/CD pipeline, and was used early into development for the purposes of viewing the original ABET App.[16]

Docker: An open-source platform that allows for applications and their dependencies to be containerized into standard units. This allows us to have a simple way to ensure that all components of the project work on any machine and are easily deployable to production.

Liquibase (4.33.0): Database migration software that consists of a master changelog and changelog files. Each changelog file represents a new update/addition/removal to the database. This allows database changes to be organized neatly with clearly defined rollback procedures.

Nginx: An open-source web server, reverse proxy, and HTTP cache. This technology serves frontend content directly, creating faster load times as well as caching frequent backend requests reducing response time.

Github Pages: Included with Github, Github Pages is a tool that allows static Html pages to be hosted and viewed directly without needing to be hosted on a personal computer.

Bucket4j: This is a rate-limiting library for Java. It is used to implement rate limiters on web applications which prevent the API from being overwhelmed by a single source.

Design

High-Level System Design

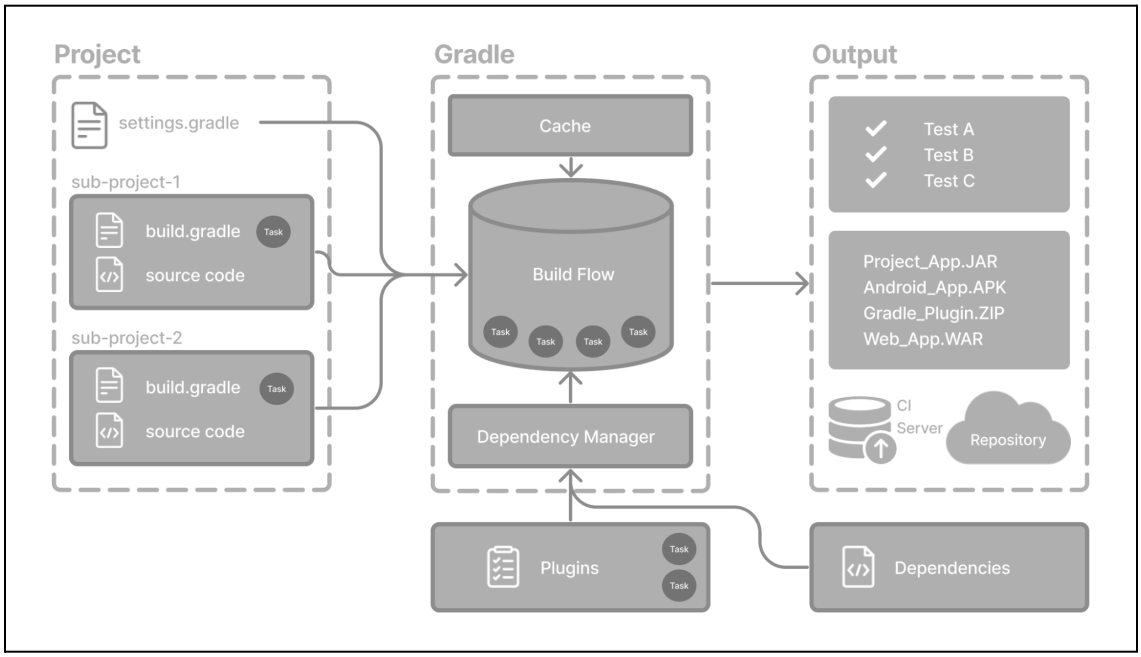


Figure 3. Gradle System Architecture

The system was designed with best practice coding standards in mind, making sure concepts such as DRY (Don't Repeat Yourself) and clear, small functions are prevalent throughout the application. Not only is this good practice, but it also means that developers spend more time working on new features, rather than recreating existing functionality in each file. The backend architecture was designed around a set of base classes, which take common functionality from the child classes that inherit from them. For example, the base controller class has methods for error handling, creating pageable responses, and a health check endpoint. This reduces the amount of repeated code throughout the backend, as certain annotations and functions can be kept in the base classes. The backend also has a clear division of responsibilities, with each layer of the application being designed for a specific purpose.

The frontend makes even better use of these principles, as entire components of the UI are able to be used throughout the application. This allows for elements such as cards, modals, or inputs to be kept stylistically identical. It also cuts down on repeated code by ensuring that developers are not creating the same components over and over again inside the larger encompassing components, like pages. Having the components function in this way also allows for changes made to the components to cascade throughout the rest of the application, so a developer does not have to update every individual element of the UI.

Database Schema

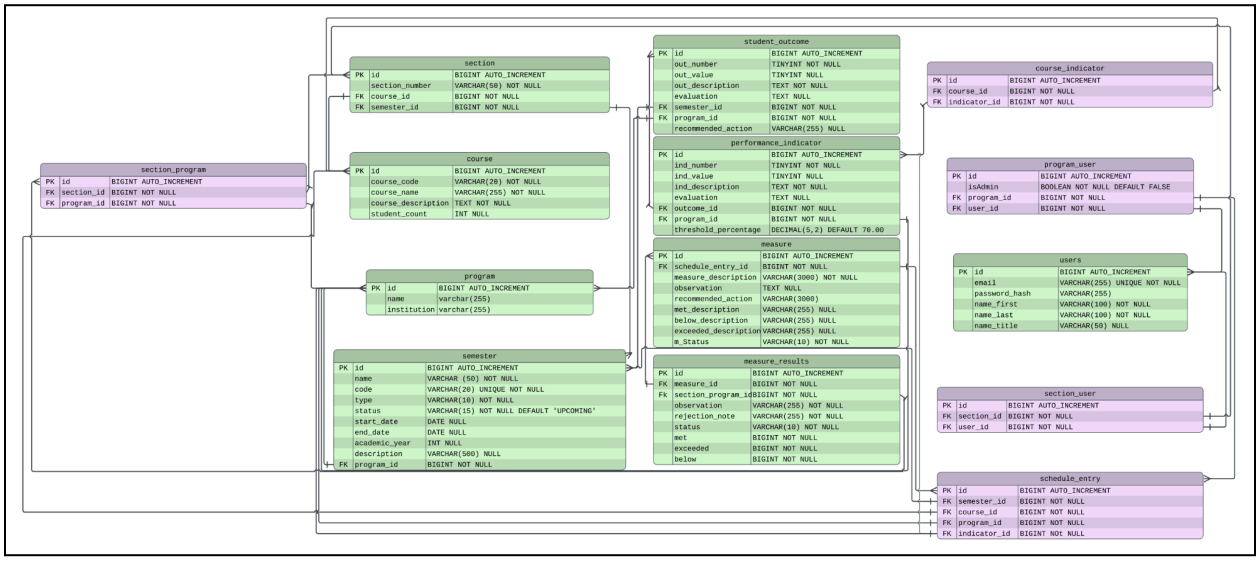


Figure 4. Database schema diagram

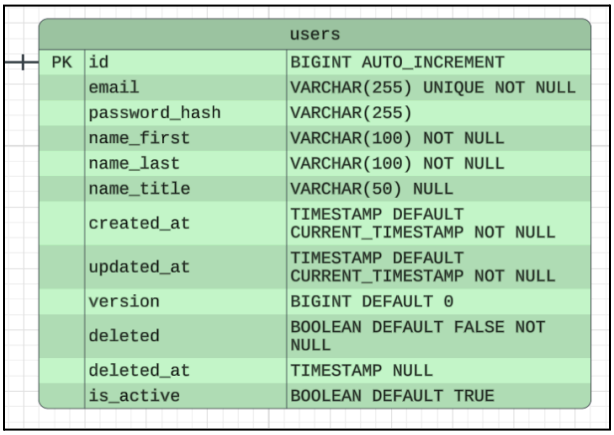


Figure 5. Automatic extra database table fields

The database chosen to support this ABET application is MariaDB. This database design centers on modeling the relationships between key assessment components such as student outcomes, performance indicators, and measures and connecting them to the institutional structures in which they are evaluated such as programs, courses, sections, and semesters. In the current system there are also two implemented user roles: Instructor and Admin. Each of these entities plays a role in contextualizing assessment data. The relationships between these entities are enforced through foreign keys and structured to maintain referential integrity, ensuring that all assessment data remains consistent and accurately linked.

The design also accounts for workflow requirements, particularly the need to input, review, and manage assessment data. By structuring the database around both assessment relationships and institutional hierarchy, the system supports typical academic processes such as section-level assessment entry, program-level aggregation, and administrative oversight. This dual focus allows the database to function not only as a storage system but as a foundation for reporting and decision-making.

To manage the evolution of the database schema over time, the system utilizes Liquibase changelogs. Instead of manually modifying the database, all schema changes, such as creating tables, altering relationships, or adding constraints, are defined in version-controlled changelog files. Each changelog represents a discrete, incremental update to the database, allowing changes to be applied consistently across different environments. This approach ensures that the database structure remains synchronized with the application code while also providing a clear history of modifications. By incorporating Liquibase into the design, the system supports maintainability, reproducibility, and team collaboration, reducing the risk of inconsistencies or errors during development and deployment.

Backend Design

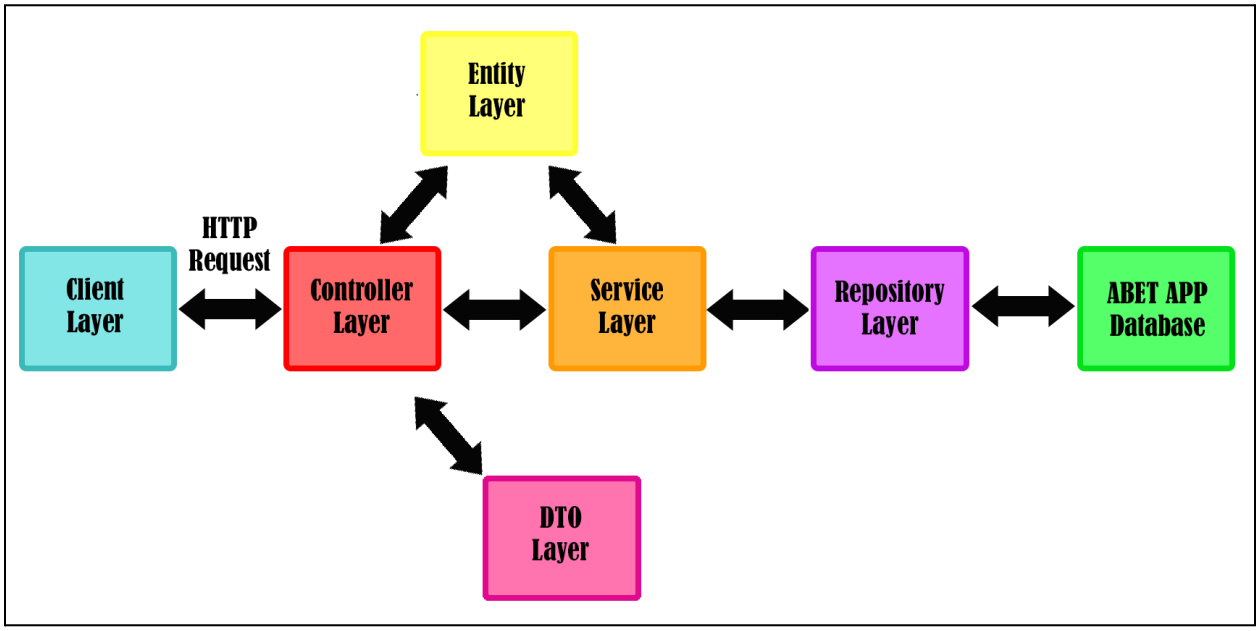


Figure 6. Spring Boot Architecture Flow

The backend of the ABET App is divided into six distinct layers; controller, service, repository, entity, DTO, and database. The relationship between each of these layers is shown above in Figure 6. The “client layer” included within Figure 6 is a stand-in for the frontend. The database layer is where the schema, and by extension all of the data, is located. The service layer can read or write to the database layer with the use of the entity layer and the repository layer.

The repository layer provides the service layer with methods to query the database layer, and the entity layer gives the service layer a place to store data going to or from the database. The service layer communicates with the controller layer, to either send or receive data with the use of the DTO layer and the entity layer. Finally, the controller layer communicates directly with the frontend, to either display the data gathered from the service layer, or to send data to the service layer to have it saved to the database layer.

The entity layer contains entity classes. These entities are used for storing data pertaining to specific tables within the schema. Every table in the schema has an entity corresponding to it, and these entities contain constraints to prevent invalid data from being incorporated into the database. With the exception of the “is_active” field, each of the “base fields” shown in Figure 5 are stored within a separate entity on this layer called the “BaseEntity”. The DTO layer contains DTO classes, which stand for Data Transfer Object. Similarly to the entity layer, there is a DTO for every table in the schema. DTOs contain only the necessary fields to pass data from the API into the service layer. DTOs are often used for updating entries.

The repository layer contains repository classes. Each of these classes contains methods for directly querying the database. These queries can perform CRUD operations in the database, as well as gather more specific information such as returning a boolean that indicates whether or not a specific table entry exists, or counting the number of entries that exist within a table under specified conditions. Similarly to the model layer, there exists a repository for each table in the schema, and there also exists a “BaseRepository” which contains methods that are useful for every repository class in the project.

The service layer contains service classes. Each method stored within a service class will call a method stored within a repository class at least once, but it’s common for several different repository methods to be called by one service method. There exists a service class for each content table in the schema, but not the junction tables. Instead, the junction tables are integrated into whichever classes they are needed in. For example, the course service contains methods for retrieving course indicators. There also exists a BaseService for the other services to inherit common functionality from. Finally, there is the controller service, which contains the controller classes. Controller classes contain methods that are each tied to a specified URL, and call methods from the service layer. For every service that exists within the project, there’s a corresponding controller.

An example of this functionality would be calling the login endpoint in the users controller. The data from the request, which would include the username (in this case an email address) and the hashed password, is put into a DTO by the controller. The controller then maps the fields in the DTO to the corresponding fields in the entity, which is passed to the service. The service will then use the data in the entity to check against the entries in the database. This is

accomplished by calling the repository class, which in turn queries the database. After the username and password have been verified, the service lets the controller know that the login is valid, and the controller sends a response to the frontend to allow the user to login.

Frontend Design

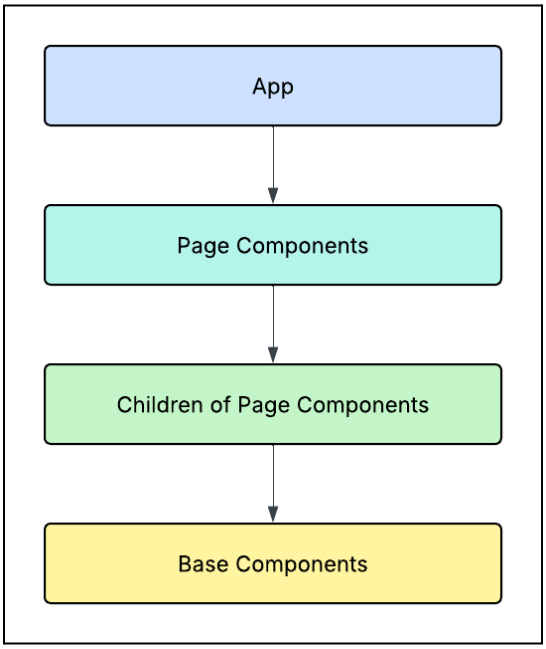


Figure 7. Frontend Component Hierarchy

As shown in the UML Diagram in Figure 7, the frontend components are organized with a complex web of parent-child relationships. These relationships can be generalized into the component hierarchy shown in Figure 7.

At the very highest level of the Vue component hierarchy is the App component. Every other component is a child of the App component. The App component contains the Navigation Bar component and the main router view. The router view is an element in Vue that changes which page component is displayed based on the content of the URL. For example, the Home Page is accessed by adding “/home” to the end of the URL. When the router sees this, it will update the router view component to fetch the Home Page component. The router can also be utilized through the router link tag, a Vue tag that acts as an HTML link tag but can be used to navigate to a different page on the site.

The lowest tier of the component hierarchy (Figure 7) are the base components. These components serve as the building blocks for every other component in the frontend. Base components are designed to be generic elements that can be reused in different contexts throughout the frontend. These components include buttons, inputs, modals, cards, and toasts.

There are several advantages to using base components. Reusing common generic components prevents code from being needlessly rewritten, while also maintaining a consistent visual style throughout the application.

Pages are designed to be the largest meaningful divisions of the full functionality of the app. Each page is designed to contain information and functions that pertain to a particular aspect of the site. For example, the Home Page serves as the informational and navigation hub for the site, while the Section View Page contains all functionality related to a specific course. Pages are designed to contain enough content to warrant the full space of a web page, but not so much content that users become overwhelmed or confused.

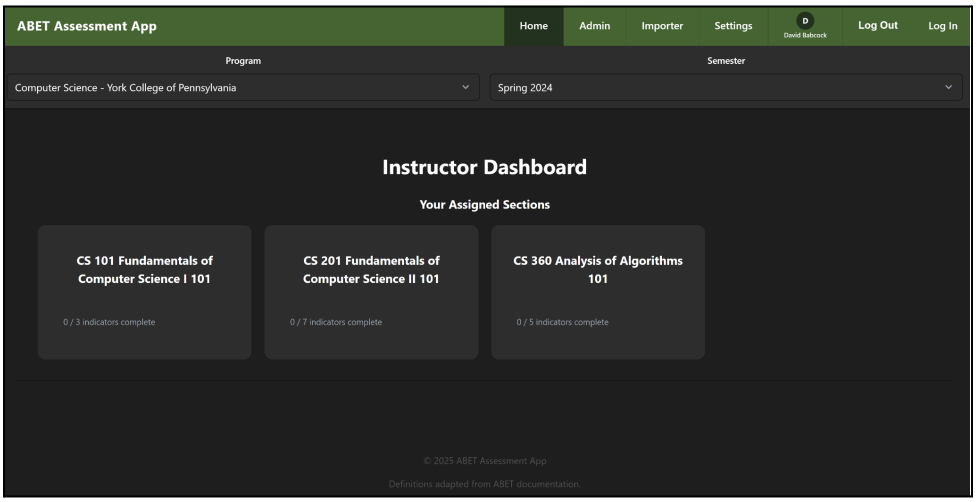


Figure 8. ABET App Home Page

The Home page (shown in Figure 8) serves as the starting point for navigation throughout the site. Each element of the Home page is designed to show the user basic information and help them navigate to the right part of the application to perform their required tasks. The top of the home page contains the Program and Semester selectors. These will dynamically update the content for the rest of the site to pertain only to the selected program and semester. The Home page also contains the instructor dashboard. This is an element that displays a list of the current sections for which the currently logged-in user is the instructor. Clicking on these section cards navigates to the corresponding Section View page.

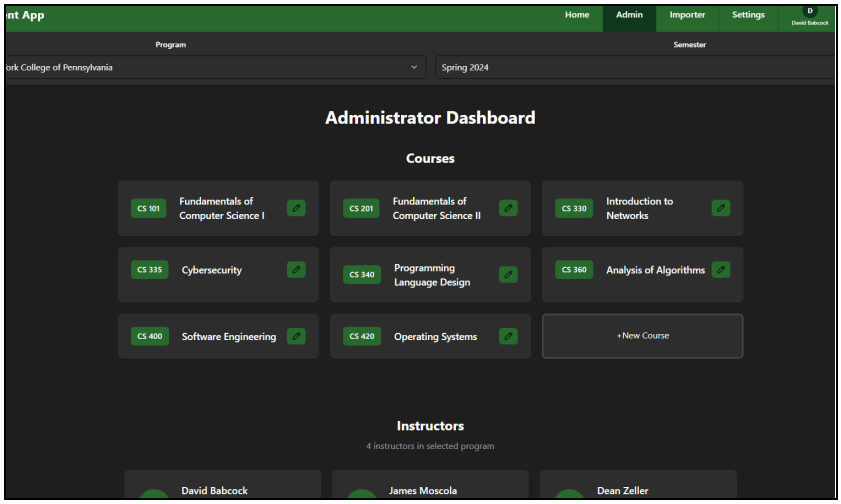


Figure 9. Administrator Dashboard

The Administrator Dashboard, as shown in Figure 9, appears on the Navigation tab as the "Admin" Page when the signed-in user has administrator privileges. This component serves as the main navigation hub for administrators, providing an overview of the entire program and containing child components that list every course and instructor in the program. An administrator can make edits, delete and create courses as well as edit and create instructors. Each course and instructor listing component links to the respective Instructor view modal or Course view modal, allowing the administrator easy access to all the resources available for the semester. This contains a list of all courses assigned to that instructor, with course listings linking to their respective Course view modal. Once you click on the courses, an administrator can see the course's corresponding sections as well as create sections for the corresponding course.

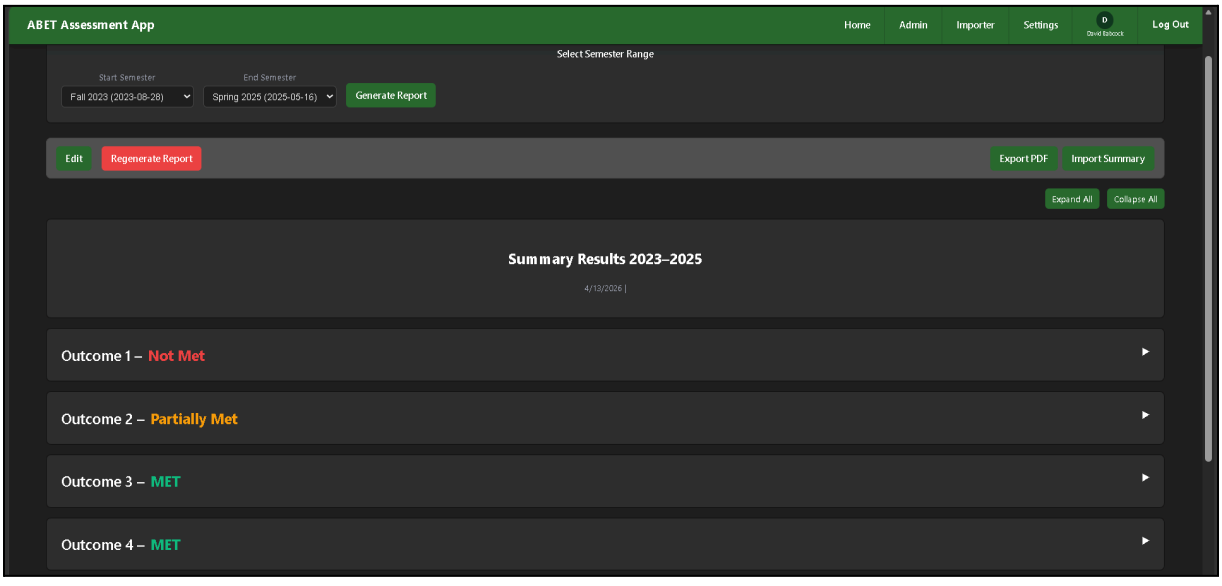
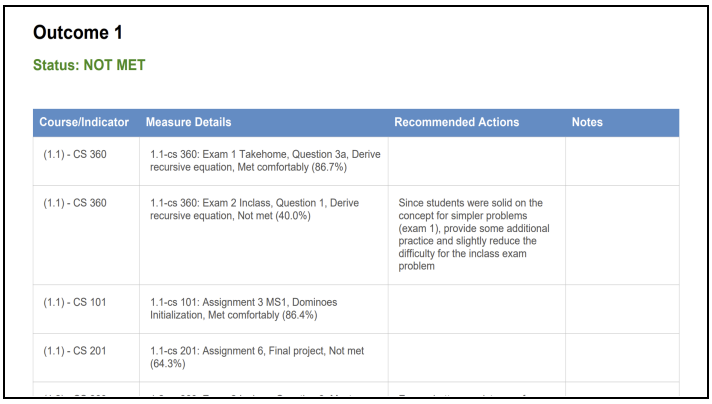


Figure 10. Summary Results Component



Outcome 1

Status: NOT MET

Course/Indicator	Measure Details	Recommended Actions	Notes
(1.1) - CS 360	1.1-cs 360: Exam 1 Takehome, Question 3a, Derive recursive equation, Met comfortably (86.7%)		
(1.1) - CS 360	1.1-cs 360: Exam 2 Inclass, Question 1, Derive recursive equation, Not met (40.0%)	Since students were solid on the concept for simpler problems (exam 1), provide some additional practice and slightly reduce the difficulty for the inclass exam problem	
(1.1) - CS 101	1.1-cs 101: Assignment 3 MS1, Dominoes Initialization, Met comfortably (86.4%)		
(1.1) - CS 201	1.1-cs 201: Assignment 6, Final project, Not met (64.3%)		

Figure 11. Summary Report PDF Export

The Summary Results component (shown in the app in Figure 10) is a component of the Admin dashboard that displays the complete results of each Student Outcome for the given semester. Each performance indicator associated with the outcome is listed, with each associated measure being listed under its respective indicator. For example, Figure 10 shows results for outcome 1, which contains performance indicator 1.1. Performance indicator 1.1 contains a measure with the description “Exam 1 Takehome, Question 3a, Derive recursive equation” and another measure with the description “Exam 2 Inclass, Question 1, Derive recursive equation.” The degree to which each measure and indicator is met is calculated based on the threshold percentage associated with each indicator. The degree to which an outcome is met is then calculated based on the number of associated indicators that are met. Each measure listing contains a router listing to the FCAR page associated with that measure. You can choose what data you want from what semesters by choosing the starting date and end date before generating the report. Once the summary results report is generated, the summary data can be exported to a PDF file (shown in Figure 11). This file is formatted to contain the outcome, performance indicator, and measure data for a full calendar year.

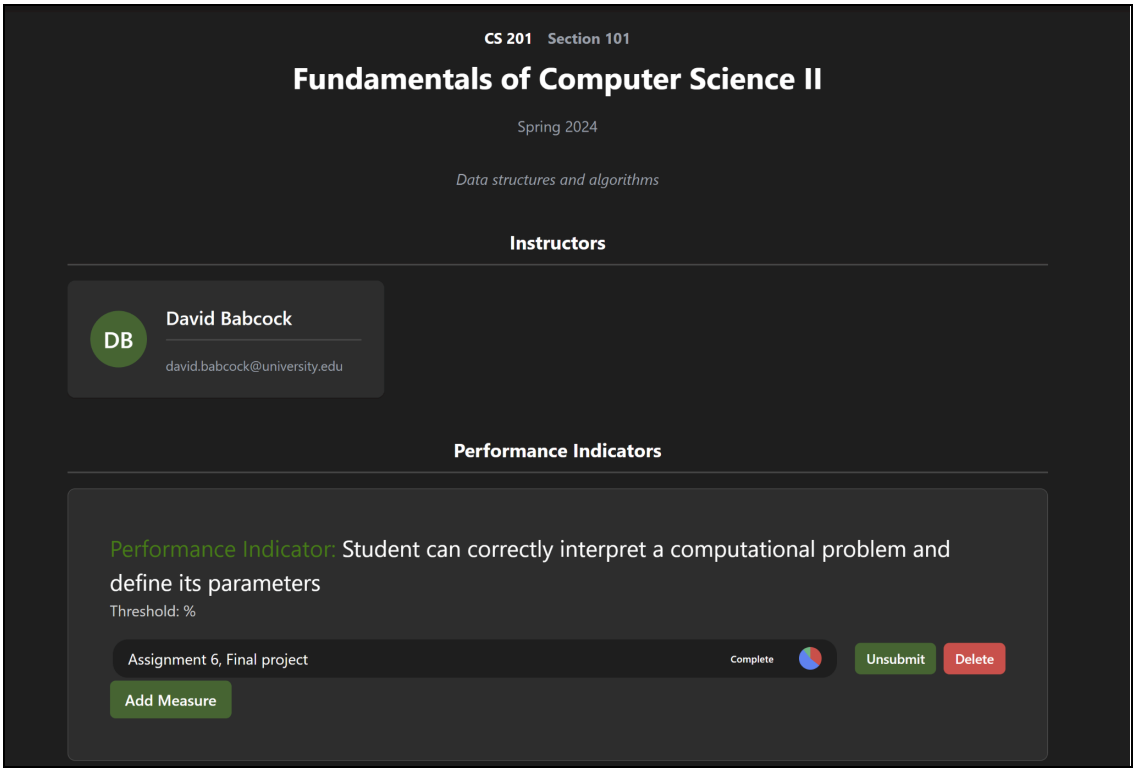


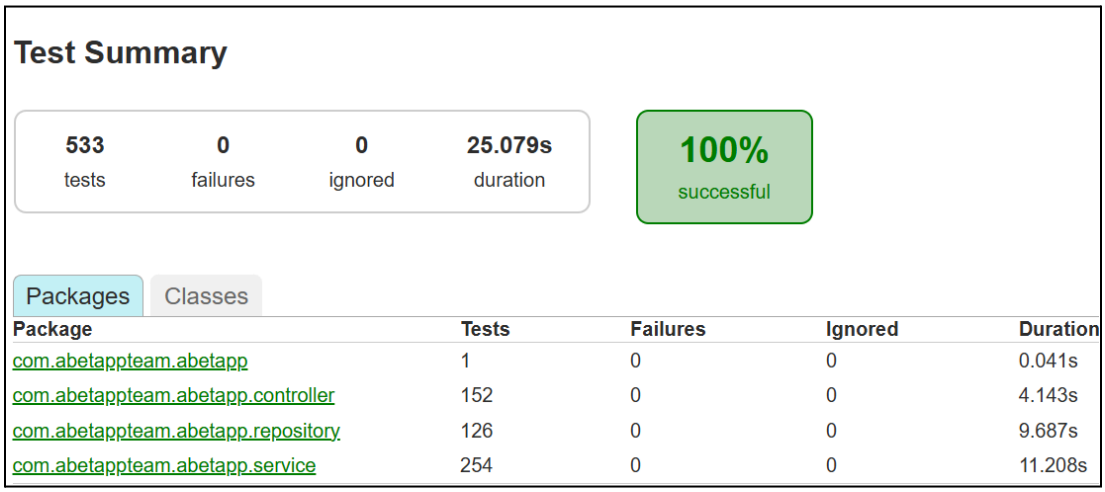
Figure 12. Section View Page

The Section View Page (shown in Figure 12) contains information and functions related to a particular course's section. The main function of this page is for instructors and administrators to define and complete measures. Measures are arranged by performance indicator. Instructors have the ability to create a new measure under a particular performance indicator. Instructors can also complete a measure by inputting how many students met, exceeded, or fell below expectations, as well as optionally submitting an observation. Instructors can also edit or delete a measure until it has been submitted. Administrators are also able to add or delete measures at any time, as well as mark an unfinished measure as complete.

Testing

Backend testing has been developed with JUnit and is built with Gradle. Testing covers every repository, service, and controller class in the backend, and contains tests for most of the methods found in that class. In total, there are 533 tests across the entire backend with ~80% coverage. Whenever tests are run, a file located under the build folder named "index.html" in the project directory updates and displays information about the tests. This information includes the number of tests that ran, which tests passed, which tests failed, how long the tests took to run, and how many tests were included in each file (shown in Figure 13).

Each layer extends a base class that provides a shared configuration and utility methods. BaseRepositoryTest uses an in-memory H2 database to verify queries and data access logic. BaseServiceTest uses Mockito to mock dependencies and test business logic without needing a database connection. BaseControllerTest uses MockMvc to test HTTP request handling and response codes without starting a server. A TestSecurityConfig class disables JWT authentication across all controller tests so that endpoints can be tested without requiring valid tokens. This base class approach ensures that tests are consistent across the entire test suite and reduces duplicate code.



Test Summary

533	0	0	25.079s	100%
tests	failures	ignored	duration	successful

Packages | Classes

Package	Tests	Failures	Ignored	Duration
com.abetappteam.abetapp	1	0	0	0.041s
com.abetappteam.abetapp.controller	152	0	0	4.143s
com.abetappteam.abetapp.repository	126	0	0	9.687s
com.abetappteam.abetapp.service	254	0	0	11.208s

Figure 13. Index.html test summary

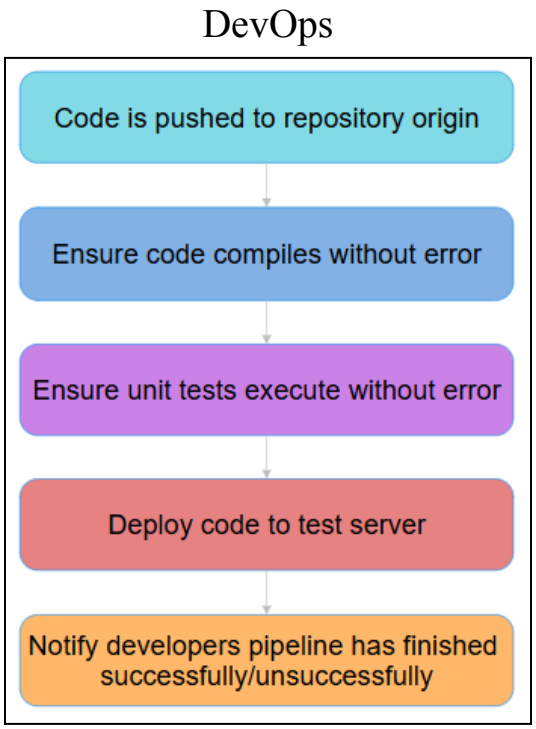


Figure 14. CI/CD pipeline workflow

Having a CI/CD (Continuous integration, continuous development) allows for developers to be more accountable about the code they write as well as easily monitor how their code compiles, tests, and deploys to ensure proper precautions are taking place when setting up production environments. As shown in Figure 14, the CI/CD pipeline implementation for ABET app's codebase allows the development team to quickly see any issues present in pushed code. If unit tests are failing, it is easy to tell where and what tests need attention. A notification system is also present that sends out a message to the development team whenever a pipeline finishes as well as the status it finished with. This not only allows individual developers to correct mistakes quicker, but also increases accountability within the development team.

Containerizing software applications allows for code to be executed in any environment, regardless of the software on the system. This makes developing easier among a team, as there can be confidence that if the container runs on one machine it will run on all of them, but this also makes deploying an application much easier. The container implementation used for this project is Docker. Having a container for each aspect of the project allows for developers to easily monitor output logs for the frontend, backend, and database to easier isolate problems.

With this improvement to the organization of the project, having software to manage and log new database changes while providing rollback procedures if new database changes were to fail would continue to make development more straightforward. Having a Liquibase

implementation in the project provides these features. With it, developers can create new changelog files, and add them to a master file. This gives the rest of the development team a clear log of who made what changes. While improving organization, this helps reduce potential merge conflicts when multiple developers edit the database at the same time, as they would be working in separate changelogs.

Implementation

Database

The database schema was a major point of focus throughout the development of the ABET App over the course of the Spring 2026 semester. While the previous implementation was sufficient for a basic proof of concept, it had to be refactored several times to account for edge cases that arose during practical use.

One of these edge cases is the existence of multiple sections of the same course existing in the same semester. This prompted the creation of a separate Section object that stores data unique to an individual section, while linking to a Course object that contains information common to all sections of that course. A similar solution was made when considering that measures are shared between all sections of a course for a semester, but the measure result data needs to be recorded individually. The solution to this issue was to create a Measure Results object that stores a record of how many students from a particular section met, exceeded, or were below expectations for a specific measure.

This refactored database also implemented and utilized major junction tables to handle real workflow functionality needs, such as the Section User, Schedule Entry, and Section Program tables. The Section User table handles the more simple case of when there may be more than one instructor teaching one section of a course. The new Schedule Entry table handles a much more complex relationship. It includes the IDs of a Performance Indicator, Program, Course, and Semester. Performance Indicators for a program are mapped to be evaluated during specific semesters and courses before any course sections are made. The Schedule Entry table holds this data and is used to pre-populate sections with which performance indicators it will be expected to evaluate, taking into account the programs of the students who will be entering that section. If there are multiple programs to be evaluated in one section it is stored in the Section Program table. These new tables hold the relationships that are vital to the real workflow of the ABET assessment process.

Backend

Entity classes all follow a formulaic development structure. Every entity is composed of a set of variables referred to as columns, each of which corresponds to a field in one of the database tables. Each entity includes a getter and setter for each field, constructors, and a method to convert the entity to a string. Some entities have additional methods, such as the users entity or semester entity, but these differences are rare. The most common differences between each entity are the constraints. Each column needs specific constraints applied to them to ensure that

no invalid data is passed to the database. Some of these constraints are commonly used, while others can be unique. For example, the users entity is the only class in the project which includes a constraint that only allows valid email addresses to be passed to the email column. When adding a new entity to the project, the schema was always used as reference to inform both the names of the fields and the constraints. Following the creation of an entity, the DTO corresponding to the same table would be created by copying the entity and editing it by removing some of the code. In particular, the lines denoting that variables were columns and the string method would be removed.

Repositories were often among the most challenging components of the backend to implement, as repository method definitions rely on a strict naming convention that is not validated by Visual Studio Code. Based on the name of a repository method, Spring Boot is capable of automatically inferring the desired database query. If a method was called “findById” and it had the proper input and output, Spring Boot would automatically generate and execute the corresponding database query and return the mapped result. However, if a method was called “findFromId”, this method would create an error and the IDE wouldn’t indicate that the code was wrong. Furthermore, unit tests were often the least reliable with identifying errors with the repository layer. A big push this semester was to simplify the way that new repository classes were written. In previous implementations, whenever a new search method was needed to query the database, a new repository method would be created. The issue is then a new service layer method and controller layer method would need to be created. This problem was ameliorated by removing all of our pre-existing search methods in favor of having a single generic search for each table. This search would contain all the fields that might need to be searched on for a given table with those parameters being excluded if no value is passed for a given search parameter. This significantly cut down on not only repository methods, but also service and controller layer methods as well.

Creating test methods was often one of the most time-consuming aspects of backend development, but it is critical to ensure that the classes being developed have the expected functionality. Whenever a test failed, it was possible that either the test did its job and an error was just caught prior to any faulty code being committed to the project, or the test was simply written incorrectly. Testing was an invaluable tool to the team at large, and prevented poorly written code from being incorporated into the project several times.

Frontend

The frontend of the application encompasses the layers of the application that the user interacts with directly. The frontend for ABET App is hosted on the web using a Vite server. The frontend framework used for the web application is Vue. Vue provides a framework based around components, which contain a defined set of frontend elements that can be re-used

throughout the application. This provides an object-oriented structure to frontend implementation.

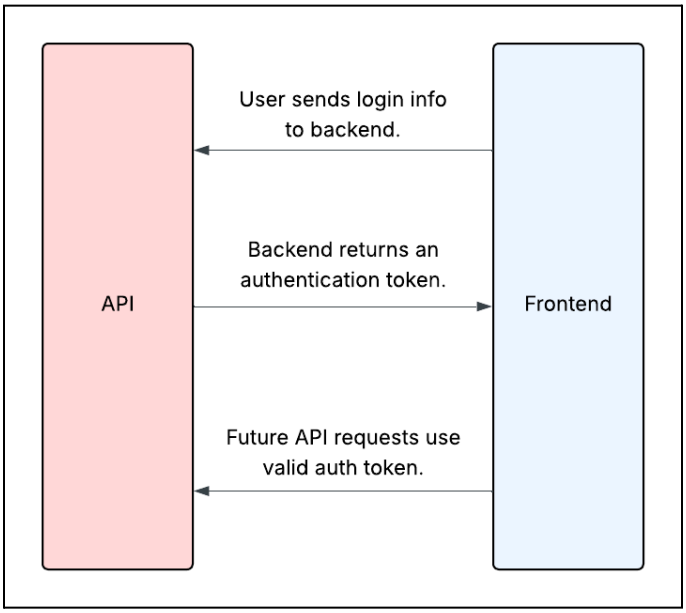


Figure 15. Authentication Token Exchange

One interesting problem encountered in frontend development was deciding how to handle user authentication. The solution was to use JSON web tokens, or JWT. The frontend makes a request to a backend API endpoint, and it returns an authorized token. This token is then used to authenticate API requests through a wrapper that attaches the token. This process is illustrated in Figure 1. The token has three parts; a header, payload, and a signature. The header contains information about the type of token and the algorithm used. The payload contains all of the app specific data, and the signature is used to ensure the data was not tampered with. The tokens have a specific expiry time set, which allows for logins to persist in-between user sessions. This system provides basic security to the data stored in the backend, as the backend will not respond to HTTP requests that do not contain a valid token.

The parent-child relationship network for Vue components is useful for abstracting frontend functionality and supporting expandable code, but this structure does have some limitations. One such limitation is that information needed across multiple components, such as the signed-in user or the current semester, must be passed from the parent component down to the child components. This requirement can become complicated and cumbersome as the application grows. To solve this issue, the frontend makes use of stores, which are frontend classes that can obtain and disperse data to any frontend component. The project makes use of the NodeJS library Pinia to create stores. The stores are used for things such as state management, access control, storing application context, and persisting data to storage. For state management, the store has fields for information such as the name, email, or ID number. The stored items are available globally, which maintains the information across all parts of the app.

Access control in the store is initialized after login, once the user's role is received. This role can be used to decide which components or pages a user is allowed to see or use. The store also holds information relevant to the application, such as the currently selected semester or program. This context is also important to properly displaying information to the user, and when that information is updated by a component, it propagates throughout the application, as all the components reference the store. The last main functionality of the store is to persist data to local storage. It is important to not store any sensitive data in the local storage, but context like the selected semester or program can be stored between sessions to keep the user experience consistent between sessions.

DevOps

The DevOps aspect of the application covers multiple different processes throughout the project. For this semester, DevOps was focused on CI/CD, project building, containerization, deployment, security, and documentation. The purpose of these projects is to improve the development experience for current and future development teams and to set the project up for public deployment and domains when proper functionality is achieved.

Version control for the project was performed using Git and the repository origin is located on GitHub. This being the case, GitHub has a specific format for how CI/CD pipelines operate. GitHub provides a folder when creating a repository that allows developers to create "workflows". It is these workflows that allow the root repository to run tests, build code, and deploy code. These workflows are written as YAML files, containing mostly human readable text. For the implementation in the project, two workflows were created. These workflows were given the context to run whenever the "push" action was performed locally. When pushed, GitHub provides execution environments for these actions to run in, which must be specified by developers. For the purposes of this project, the workflow is running in Ubuntu on the latest version. The first action does the "gradle clean" action, which clears out the build folder if, potentially, it had any contents. Then the workflow runs "gradle assemble" which is one of Gradle's provided build commands. The "assemble" command here was chosen over the more conventional "build" for a few reasons. The first reason being that "assemble" does not run any unit tests when compiling. This is important as having our unit tests run separately from the build will help improve readability when errors arise. The second reason is that "assemble" will create an executable JAR file for a java project. This essentially simulates the processes of creating a runnable web server file, which is the proper check to perform here. Simply just checking that the codebase compiles is not sufficient. After this process is complete, the workflow will then ensure that all the unit tests execute without failure. If at any point in the process a step fails, that failure is noted and the workflow moves to the next action. Once this process is completed, a Discord bot using GitHub's provided API interaction has been set up to provide developers with

notifications of pipeline status. This ensures developers stay accountable for how their code is performing, and can rapidly respond to any issues that arise.

The build process had gone through many iterations over the course of the project. Initially, all of the building was handled using curated Gradle scripts that would cherry-pick what aspects of the project to build and execute. There was then a push away from using Gradle to execute the project, and build commands were created to only build the project while executing the project would be handled by the IDE. In the end, the simplest solution was to use Docker to handle building and execution. Now, the entire project can be built and deployed with a single command that re-builds and re-deploys each container. This solution seemed to work the best with the improvements that had been made to database management. This addition now lets the Liquibase implementation run every time the project is started back up, removing the redundantness of needing to run it separately from the project. In this same vein, having the project containerized has greatly improved the portability of the codebase. Using docker containers for the frontend, backend, database, and liquibase implementation, developers can now port these containers onto any machine and execute the project. This will be especially important moving forward as the project is looking to move to be open-source. The Docker implementation for this project is two-fold. There is a Docker environment for the frontend and backend. The backend environment contains the build script for the Spring backend, as well as the database implementation and Liquibase implementation. The frontend environment builds the frontend using Node and serves the content using Nginx.

Now having a Docker implementation, deploying the project has become more straight-forward. Beforehand, deployment was being attempted using a convoluted process where a script would SSH into the deployment server and use FTP to transfer the compiled server files to execute. While this process did work, it was much too cumbersome and unreliable. This is where the Docker implementation comes in. With the project all being containerized, the only thing that needs to happen for the project to be deployed is new versions of the containers to be pulled from the remote. The Docker environments are set up in such a manner that when the code is executed on the server, a port is exposed that general internet traffic can connect to through the server's IPv4 address. This has greatly reduced the amount of environment variables and Spring configuration that needs to be performed in order to have the project deployed.

Since the project has now been deployed to the web, security starts to become more important. Previously, fairly standard security measures had been implemented in the application to prevent from basic attacks. API requests require authorization tokens and text input fields don't allow for malicious data entry, like SQL injection. But since the project is exposed to the internet, improvements to the general security would greatly improve stability. The first addition was a reverse proxy server that was created using Nginx mentioned previously. This implementation creates a reverse proxy server that helps balance the server load by regulating

traffic and ensures that connections are not able to view any private information about the server, such as the IP address. A rate-limiter was also implemented to help prevent malicious actors from attempting to force their way into the system. The rate-limiter implementation was created using Bucket4j, which provides each user with a set of API tokens that refresh on a timer. The current rate-limit is set to one-hundred requests per-minute. When this limit is reached, the software will drop the connection with the user, while allowing other users to still connect and use the system.

In terms of documentation, since no previous standard existed, a documentation website was created. This is meant to be a living document that contains all relevant information to the project. In its current state, it contains information about project terminology, setup instructions for new users, and an explanation of what the project is. As said before, this is meant to be a living document that each development team continues to add on to as the project nears completion.

Future Work

Future Backend Work

As the system evolves, improvements to the backend architecture will focus on better alignment between Spring Boot and Liquibase in order to streamline database management and fully leverage the strengths of both technologies. In the current implementation, there are overlapping responsibilities between application-level configuration and database schema management. Certain aspects of the schema are influenced or inferred by Spring Boot's ORM layer (entity mappings and auto-generation behaviors), while others are explicitly defined and version-controlled through Liquibase changelogs. This dual approach can lead to inconsistencies in how the database is created, updated, and maintained across different environments. Moving forward, the system should more strictly delegate all structural database changes, such as table creation, constraints, and relationships, to Liquibase, while limiting Spring Boot's role to object-relational mapping and data access. This would eliminate ambiguity caused by features like automatic schema generation and ensure that all database changes are explicitly tracked, versioned, and reproducible.

Another area for future work is account recovery. The current system does not provide any way for users to reset their passwords or regain access to their accounts if they lose their credentials. Implementing a secure account recovery would involve generating reset tokens, sending recovery emails, and allowing users to set a new password through a verified link. This is an important feature since, without it, administrators would need to manually intervene whenever a user is locked out.

The backend would also benefit from a notification system to keep users informed of important events within the application. For example, instructors would receive a notification when they are invited to join a new program, and again when a measure they are responsible for is reaching its due date. Administrators would also be notified when an instructor has completed a measure. These notifications would be delivered through in-app notifications.

Future Frontend Work

The progress on the frontend of ABET app has provided a strong foundation for future development. However, there are some issues present in the current frontend implementation that must be addressed in future work. There are also some remaining frontend features that need to be implemented before the project can be finished.

The current implementation has some pressing issues regarding the instructor initialization process. In its current state, administrators are required to create the username and password for each instructor to be added to their program. This makes it impossible for a user to be added to multiple programs. This implementation should be replaced with a system where user accounts are created without regard to the program, either by the instructor themselves or by an administrator. Instructors should then be invited to programs using their email address.

Future implementations should include a button instructors can use to mark a performance indicator as complete. This is used to keep track of the accreditation progress for a section. At the end of a semester, the administrator should look through the completed measures and assign recommended actions if needed. Administrators should also evaluate whether or not each outcome was sufficiently met by the student body. This can be accomplished with a workflow, allowing administrators to go over the year's accreditation data outcome by outcome and assigning recommended actions and evaluations.

Another feature to be added is the Indicator Schedule. This will be where administrators can determine ahead of time which indicators should be evaluated for each course in any given semester. This is an integral part of the accreditation process, as which performance indicators are evaluated for a given course vary per semester. The Administrator Dashboard should link to an Indicator Schedule page, where administrators can define and edit the indicator schedule for upcoming semesters. This will add schedule entry objects to the database, which are used throughout the application to load indicators for sections.

Another frontend feature to be added to the application before release is notifications. Users should be notified when they are invited to a new program by an administrator. Instructors should be notified when they have unfinished measures or performance indicators. Unread notifications should be indicated in the navigation bar. A new Notification page should be accessible via the navigation bar. The Notification page should display a list of the current notifications associated with the logged in user.

Beyond these major features, there are some other smaller updates that should be implemented before release. There is currently no way to initialize semesters in the front end. This could be done manually by administrators, but could also be done automatically based on the current date. Measures should include rubrics that define what constitutes a student's performance being below, having met, or having exceeded expectations. After having completed the outcome evaluations for a calendar year, the summary report should become "official" and look visually distinct from an incomplete, "unofficial" report. Lastly, administrators should have the ability to export an FCAR document for every measure in a semester at once.

Future DevOps Work

The progress from this semester has placed the application in good standing with DevOps technology, but there are some aspects that need attention still. Namely, security is still a large concern. Research needs to take place for what common security vulnerabilities exist in the tech that the project uses and how to combat them. Since this system contains lots of important, sensitive information, security is paramount. Once security is improved, proper deployment would be the next step in improving the CI/CD pipeline. Currently, deployment is done manually as the developers wish to keep a close eye on the state of the deployed code and want to ensure that the system is secure before there is the potential for malicious users to try and connect to it.

References

[1] ABET, “Home - ABET,” “ https://www.abet.org/”

[2] ABET, “Criteria for Accrediting Computing Programs, 2025 – 2026,” “https://www.abet.org/accreditation/accreditation-criteria/criteria-for-accrediting-computing-programs-2025-2026/”

[3] Microsoft, “Visual Studio Code - The open source AI code editor,” “https://code.visualstudio.com/”

[4] OpenJS Foundation, “Node.js — About Node.js®,” “https://nodejs.org/en/about”

[5] ethomson, MylesBorins, mrienstra, lukekarrys, “About npm | npm Docs,” “https://docs.npmjs.com/about-npm”

[6] Gradle, “Gradle Build Tool,” “https://gradle.org/”

[7] Broadcom, “Spring Boot,” “https://spring.io/projects/spring-boot”

[8] MariaDB Foundation, “MariaDB Foundation - MariaDB.org,” “https://mariadb.org/”

[9] Evan You, “Vue.js - The Progressive JavaScript Framework | Vue.js,” “https://vuejs.org/”

[10] VoidZero Inc & Vite Contributors, “Vite | Next Generation Frontend Tooling,” “https://vite.dev/”

[11] Stefan Bechtold, Sam Brannen, Johannes Link, Matthias Merdes, Marc Philipp, Juliette de Rancourt, Christian Stein, “JUnit User Guide,” “https://docs.junit.org/current/user-guide/”

[12] Microsoft, “Fast and reliable end-to-end testing for modern web apps | Playwright” “https://playwright.dev/”

[13] Thomas Muller, “Quickstart,” “https://www.h2database.com/html/quickstart.html”

[14] Lucid Software Inc, “Lucidchart | Diagramming Powered By Intelligence,” “https://www.lucidchart.com/pages/landing”

[15] Red Hat Inc, “Jakarta Validation - Home,” “https://beanvalidation.org/”

[16] Tailscale Inc, “Tailscale * Best VPN Service for Secure Networks,” “https://tailscale.com/”